Manual técnico y de despliegue — Evento Cultural

Manual técnico/despliegue (artefacto 8 del marco ágil, anexo metodológico)

Roberto Riffo (frontend) · José Miguel Moncada (BD/QA) · David Herrera (backend e integraciones)

25 de septiembre de 2026

Proyecto Capstone APT122 · Duoc UC · Sede Maipú

1. Composición del sistema

Parte	Tecnología	Rol
Scraper	Python (conectores, NVIDIA NIM, checkpoints SQLite)	Recolección, normalización y enriquecimiento de eventos desde 22 fuentes configuradas (21 permanentes y 1 estacional)
Base de datos	Supabase (PostgreSQL + RLS)	Catálogo canónico, bitácora de ejecuciones y evidencia de extracción
Frontend	Angular 22 + Ionic 9 (web y móvil)	Descubrimiento: búsqueda, filtros, mapa, calendario y recomendaciones
Automatización	GitHub Actions	Ejecución periódica/manual del scraper con pruebas

Cuadro 1: Componentes del sistema.

2. Ejecución local

2.1 Scraper (Windows)

```
instalar.bat        REM crea venv, instala dependencias y Chromium de Playwright
ejecutar.bat        REM pipeline completo (extraccion + exportacion + sincronizacion opciona
python -m unittest discover -s tests -v
```

Las variables de entorno se documentan en `.env.example` del proyecto; incluyen credenciales de Supabase y el endpoint de NVIDIA NIM. **No** versionar secretos: el workflow los consume como secretos de GitHub.

2.2 Frontend

```
npm install
npm start           # ng serve en http://localhost:4200 (o ionic serve en 8100)
ng test             # pruebas (Vitest)
npx ng build --configuration production
```

En Windows existen además `install.bat` y `run.bat` para el entorno local. El frontend se conecta directamente a Supabase mediante `@supabase/supabase-js` con la clave pública anon (gobernado por Row Level Security). En caso de fallo de red o indisponibilidad remota, la aplicación activa un catálogo mock en memoria como respaldo de contingencia.

3. Estrategia de despliegue (sin Docker)

El equipo decidió **no usar Docker**; el anexo metodológico se cumple documentando esta decisión y el estado real. El despliegue se compone así:

El sistema no se aloja en contenedores: las plataformas de alojamiento y automatización son GitHub Actions, Vercel y Capacitor. El estado de cada componente:

3.1 Scraper en GitHub Actions (desplegado, en testeo)

- El workflow está desplegado y el equipo se encuentra probando el despliegue: correr el pipeline con pruebas y sincronización a Supabase, usando secretos de repositorio (nunca credenciales en el código).

Componente	Plataforma	Estado
Scraper	GitHub Actions (workflow `scrape-events.yml`)	**Desplegado, con ejecución manual y programada diaria a las 23:37** (America/Santiago; activo desde el 24-09, ajustado el 29-09; corridas #9–#13 verificadas del 25 al 29-09, evidencia en Evidencias Proyecto); la prueba dependiente del tiempo fue corregida y la corrida manual #13 terminó en verde (Plan de pruebas, sección 5.1)
Web	Vercel	**Plataforma seleccionada**: al operar el frontend directamente con Supabase (JAMstack / SPA sin backend intermedio), el despliegue es nativo en Vercel conectando el repositorio y publicando la carpeta de salida `www`
Móvil (Android)	Capacitor 8 + Android Studio	La misma web se convertirá en aplicación mediante Capacitor 8: conversión en preparación (dependencias presentes en `package.json`)

Cuadro 2: Estrategia de despliegue por componente y su estado real.

- Evidencia de ejecución: captura `Evidencia_ejecución_scraper_en _GitHub_Actions.PNG` en Evidencias Proyecto.
- La periodicidad ya está definida y activa: `schedule` con ejecución programada diaria a las 23:37 (America/Santiago) desde el 29-09 (el bloque `schedule` está activo desde el 24-09). Cada ejecución restaura el checkpoint previo, verifica secretos, ejecuta la suite de pruebas, extrae conservando checkpoints como artefactos (retención 30 días), enriquece los pendientes y sincroniza con Supabase. Para dar el despliegue por completado, el equipo debe registrar la primera corrida programada completa en verde tras la corrección de la prueba dependiente del tiempo (Plan de pruebas, sección 5.1).

3.2 Web en Vercel (plataforma de despliegue)

1. Conectar el repositorio de la aplicación a Vercel.
2. Configuración de compilación: Vercel detecta Angular o se configura:
   - Build Command: `npm run build` (o `npx ng build -configuration production`).
   - Output Directory: `www` (configurado en `angular.json`).
   - Install Command: `npm install`.
3. Enrutamiento SPA: crear `vercel.json` con reescritura hacia `/index.html` para garantizar que las rutas de Angular (`/mapa`, `/calendario`, `/auth/callback`, etc.) no retornen 404 en navegación directa.
4. Variables de entorno: las credenciales públicas de Supabase (`supabaseUrl` y `supabaseAnonKey`) residen en `environment.ts` y son seguras para el navegador bajo RLS.
5. Desplegar y registrar la URL oficial de producción como evidencia del proyecto.

3.3 De la web a la app: Capacitor (conversión en preparación)

La aplicación Android es la misma web envuelta con Capacitor; pasos de la conversión:

1. `npx cap add android` (si la plataforma no existe) y `npx cap sync`.
2. Abrir en Android Studio, compilar el APK/AAB de prueba y registrar la evidencia de la compilación.

4. Variables de entorno

Grupo	Uso
Supabase (URL y clave de servicio)	Sincronización del scraper y, en el futuro, del frontend
NVIDIA NIM (endpoint y clave)	Enriquecimiento de eventos en el pipeline

Cuadro 3: Grupos de variables de entorno (nombres exactos en `.env.example`).

5. Solución de problemas comunes

- **Pipeline detenido a mitad:** los checkpoints en SQLite permiten reanudar; volver a ejecutar `ejecutar.bat`.
- **Consola del navegador con rechazos de View Transitions:** los rechazos benignos ya se suprimen en `main.ts`; cualquier otro error de consola se trata como falla de la verificación.
- **Puerto 8100 ocupado:** liberar el proceso de `ng serve` previo.